MISSION CRÉDIT AGRICOLE · MÉMO

Kafka & microservices en 5 minutes

Les services ne s'appellent pas forcément entre eux. Souvent, ils publient des événements dans Kafka et d'autres services y réagissent.

Microservices

Le SI est découpé en petits services, un par responsabilité métier : comptes, virements, notifications, fraude… Chacun a sa base de données, son déploiement et son équipe.

Deux façons de communiquer

Synchrone · REST / gRPC

A appelle B et attend la réponse.

Ex. : le front demande un solde.

Asynchrone · Kafka

A publie un événement et continue. Les services intéressés le traitent à leur rythme.

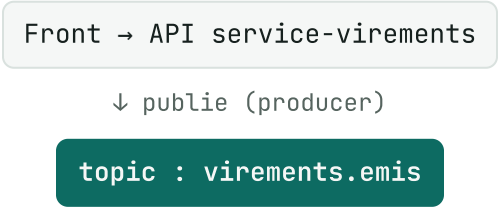
Ex. : « un virement a été émis ».

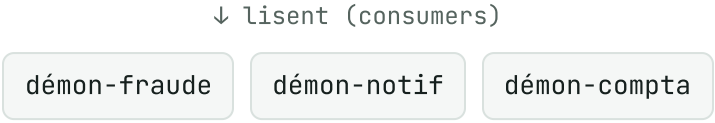
Le vocabulaire Kafka

`Event`	Un fait qui s'est produit. `{"type":"VirementEmis","montant":500}`
`Topic`	Une catégorie de messages, comme une boîte aux lettres thématique. `virements.emis`

`Producer`	Le service qui écrit dans un topic.
`Consumer`	Le service qui lit un topic.
`Consumer group`	Les instances d'un même consumer se partagent le travail. Un message est traité une seule fois par groupe.
`Partition`	Un morceau de topic, qui permet de paralléliser. L'ordre est garanti dans une partition. La clé du message (ex. `compteId`) choisit la partition.
`Offset`	La position de lecture d'un consumer, comme un marque-page. Il permet de reprendre après un crash.
`Broker`	Un serveur Kafka. Plusieurs brokers forment un cluster.
`Rétention`	Les messages restent stockés un certain temps, ce qui permet de les rejouer.
`Démon`	Un processus qui tourne en continu en arrière-plan. Ici : un consumer qui écoute un topic en boucle.

Exemple : un virement





Le service virements ne connaît aucun des démons. On peut en ajouter un sans toucher à son code : c'est le faible couplage.

Les pièges à connaître

- **Idempotence :** un message peut arriver deux fois. Le traitement doit supporter le doublon.
- **DLQ :** un topic où partent les messages en échec, pour analyse ou rejeu.
- **Schémas :** souvent en Avro, avec un Schema Registry qui fixe le format entre producer et consumer.
- **Consistance éventuelle :** les données ne sont pas à jour partout instantanément, elles le deviennent.
- **Lag :** le retard d'un consumer sur un topic. C'est l'indicateur clé du monitoring.

Mon plan pour monter en compétence

1. Récupérer (ou dessiner) la carte des services et des topics : qui produit, qui consomme.
2. Suivre un flux métier de bout en bout dans le code.
3. Repérer les outils : interface Kafka (AKHQ, Kafdrop, Conduktor), Schema Registry, monitoring du lag.
4. Lire un consumer : `group.id`, gestion des erreurs, idempotence.

Questions à poser à l'équipe

Comment gère-t-on les doublons et les messages en erreur ?

Quelle est la convention de nommage des topics ?

Comment je lis un topic en environnement de dev ?

Objectif : savoir expliquer à l'oral, en 2 minutes, un flux complet de ta mission (producer → topic → consumers).

`Mémo perso · à compléter avec les vrais noms de topics de l'équipe`